

Design a Leaderboard

Dream11 · Zynga · Riot · Duolingo · LeetCode – "Top 10 ఎవరు?" సులభం. "నా rank ఎంత?" – అదే అసలు problem.

మూడు విరుపులు. మొదటిది ఒక్క read కి **51 ms** తీసుకుంటుంది. రెండోది ప్రతి score update కి **2,00,560 మూలకాలని** కదిలిస్తుంది. మూడోది – ఏమీ మారకుండానే **leaderboard ని తారుమారు చేస్తుంది**.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి సంఖ్య నిజంగా `node` lo run చేసినదే.

ఈ DOC LO ఒక పద్ధతి మారింది - మరియు అది ఎందుకే చెప్తాను

మిగతా docs lo నేను **మిల్లీసెకన్లు** కొలిచాను. ఇక్కడ ఒక చోట అది **పనిచేయలేదు** - V8 యొక్క `splice` సమయాలు ఒకే code కి 28 ms నుంచి 1,861 ms వరకు ఊగాయి (§7). GC, JIT, memory layout - ఏదో ఒకటి.

కాబట్టి అక్కడ గడియారానికి బదులు **పనిని లెక్కించాను** - "ఎన్ని మూలకాలు కదిలాయి?". ఆ సంఖ్య **ప్రతిసారీ అక్షరాలా ఒకటే**, మరియు అదే నిజమైన ఖర్చు.

ఇది ఒక పాఠం: benchmark అస్థిరంగా ఉంటే, దాన్ని పదేపదే నడపొద్దు - **వేరే విషయాన్ని కొలవండి.**

విషయ సూచిక (Table of Contents)

Part 1 - సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి - మరియు ఇందులో రెండు వేర్వేరు ప్రశ్నలు
2. Clarifying questions - ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 - మొదటి విరుపు: ప్రతిసారీ sort

3. Step - ఒక Map, ప్రతి చదువుకీ sort
4. మొదటి విరుపు - ఒక్క "నా rank ఎంత?" కి 51 ms
5. Step - జాబితాని ఎప్పుడూ క్రమంలోనే ఉంచడం

Part 3 - రెండో విరుపు: ఇప్పుడు రాయడం విరిగింది

6. Step - గడియారం ఇక్కడ అబద్ధం చెప్పింది
7. రెండో విరుపు - ఒక్క submit కి 2,00,560 మూలకాలు
8. Step - Fenwick tree · score ప్రకారం లెక్కించడం

Part 4 - మూడో విరుపు: సమాన scores

9. Step - ముగ్గురికీ 1500 · ఎవరు ముందు?
10. మూడో విరుపు - ఏమీ మారకుండానే స్థానాలు మారాయి
11. Step - ఎప్పుడు చేరుకున్నాడో గుర్తుపెట్టడం

Part 5 - పూర్తి system

12. Step - "నేను ఎక్కడ ఉన్నాను" · మరియు నేనే మళ్ళీ చేసిన తప్పు
13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 6 - Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటీతో చెప్పాల్సిన English script
16. Follow-ups – Redis, కాలపరిమితి, విభజన
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇందులో రెండు వేర్వేరు ప్రశ్నలు

"Design a leaderboard for an online game. Players submit scores; the system should show the top players and tell a player their rank."

ఇందులో రెండు ప్రశ్నలు ఉన్నాయి, మరియు అవి పూర్తిగా వేర్వేరు కష్టాలు:

రెండు ప్రశ్నలు • ఒకటి సులభం, ఒకటి కాదు

"Top 10" మరియు "నా rank" – ఇవి ఒకే ప్రశ్న కాదు

1 • "Top 10 ఎవరు?"

10 మందిని చూపించాలి – క్లబ్ మందిలో నుంచి
ఒక heap సరిపోతుంది • $O(n \log k)$
ఇది ఒక సుపరిచితమైన problem
చాలా మంది ఇక్కడే ఆగిపోతారు

2 • "నా rank ఎంత?"

నా కంటే ఎక్కువ score ఎంత మందికి ఉంది?
Heap దీనికి పనికీరాదు – heap top ని మాత్రమే తెలుసు
Sorted జాబితా చదవడానికి బాగుంది,
కానీ రాయడానికి కాదు (§7)

మరియు మూడో ప్రశ్న – ఎవరూ అడగనిది

ముగ్గురికీ సరిగ్గా 1500 points ఉంటే – ఎవరు ముందు? ముగ్గురికీ ఏ rank?
ఇది ఒక సాంకేతిక ప్రశ్న కాదు – ఇది ఆట యొక్క నియమం.
మరియు దీన్ని నిర్ణయించకపోతే – leaderboard కారణం లేకుండా తారుమారవుతుంది (§10),
మరియు ఆటగాళ్ళు మోసం జరిగిందని ఫిర్యాదు చేస్తారు.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
"Top 10" ఎక్కువ, "నా rank" ఎక్కువ?	Top-10 మాత్రమే అయితే heap చాలు. Rank కావాలంటే పూర్తిగా వేరే structure
ఎంత మంది ఆటగాళ్ళు? Scores ఎంత తరచుగా మారతాయి?	§4 మరియు §7 – ఇదే రెండు విరుపులని నిర్ణయిస్తుంది

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
Score పరిధి ఎంత? (0-100? 0-10 లక్షలు?)	§8 – Fenwick tree పరిమాణం దీని మీదే ఆధారపడుతుంది
సమాన scores ఉంటే ఏ rank?	§10 – ఇది మీరే లేవనెత్తాలి
Score పెరుగుతుందా, లేక కొత్తది పాతదాన్ని తుడిచేస్తుందా?	"అత్యుత్తమ score" నా "తాజా score" నా – వేర్వేరు నియమాలు
రోజువారీ / వారపు leaderboard?	అవును అంటే పలు leaderboards + కాలపరిమితి (§16)
ఒక ఆటగాడిని తీసేయాలా (ban)?	<code>remove</code> కావాలి – చాలా మంది మర్చిపోతారు

మొదటి ప్రశ్న ఎందుకు అంత ముఖ్యం

"Top 10 ఎక్కువగా అడుగుతారా, లేక 'నా rank ఎంత?' ఎక్కువగా అడుగుతారా?"

నిజమైన ఆటల్లో జవాబు రెండేది, మరియు చాలా ఎక్కువ తేడాతో. Top 10 అంటే పది మంది; **మిగతా కోటి మంది తమ rank చూసుకుంటారు.**

మరియు ఇక్కడ చాలా మంది అభ్యర్థులు తప్పిపోతారు: వారు వెంటనే **min-heap of size K** అంటారు – అది top-K కి పార్యపుస్తక జవాబు. కానీ heap కి **"ఈ ఆటగాడు ఎక్కడ ఉన్నాడు" అనేది అస్సలు తెలియదు.** అది సరైన సాధనం, **తప్పు ప్రశ్నకి.**

Part 2 – మొదటి విరుపు: ప్రతినాంకం sort

3. Step – ఒక Map, ప్రతి చదువుకీ sort

అతి సహజమైన మొదటి version:

```
class Leaderboard {
  constructor() { this.scores = new Map(); }      // player → score

  submit(player, score) { this.scores.set(player, score); }

  #sorted() {
    return [...this.scores.entries()].sort((a, b) => b[1] - a[1]);
  }
  top(k) { return this.#sorted().slice(0, k); }
  rank(player) {
    const list = this.#sorted();
    return list.findIndex((p) => p === player) + 1;
  }
}
```

మరియు ఇది సరిగ్గా పనిచేస్తుంది:

```
Top 3:
  asha      1850
  meera     1500
  ravi      1200

rank('ravi') : 3
rank('asha') : 1
```

`submit` అనేది ఒక `Map.set` – తక్షణం. `top` మరియు `rank` కూడా సరైన జవాబులే ఇస్తున్నాయి. Code ఐదు పంక్తులు.

మరి సమస్య ఏమిటి?

4. మొదటి విరుపు – ఒక్క "నా rank ఎంత?" కి 51 ms

ఆటగాళ్ళ సంఖ్య పెంచి, 200 reads కి పట్టే సమయం కొలుద్దాం:

```
200 reads (సగం "top 10", సగం "నా rank ఎంత?")
```

```
ఆటగాళ్ళు | 200 reads | ఒక్క read | 10,000 reads అయితే
```

1000	22 ms	0.11 ms	1.1 సెకన్లు
20000	659 ms	3.29 ms	33.0 సెకన్లు
200000	10236 ms	51.18 ms	511.8 సెకన్లు

మూడు runs: 19-22 ms, 659-678 ms, 9,982-10,517 ms. చివరి వరుస $\pm 3\%$ ఊగుతుంది – ఇంత పెద్ద సమయాలకి అది చిన్నదే, కాబట్టి ఈ పట్టికని నమ్మొచ్చు. §6 lo అలా కాదు.

ఒక్క READ యొక్క ఖర్చు

ప్రతి read ప్రతిసారీ మొత్తం జాబితాని sort చేస్తుంది

1,000 ఆటగాళ్ళు 0.11 ms

20,000 3.29 ms

2,00,000 51.18 ms – ఒక్క ఆటగాడు తన rank అడిగితే

సెకనుకి 10,000 ఆటగాళ్ళు rank అడిగితే – అది **8.5 నిమిషాల** పని.

సెకనుకి కాదు. **ఒక్క సెకను** traffic కి 8.5 నిమిషాలు.
అంటే server మొదటి నిమిషంలోనే కుప్పకూలుతుంది.

మొదటి విరుపు – ప్రతి చదువుకీ మొత్తం పని మళ్ళీ

`rank('ravi')` ఒక్క సంఖ్య ఇస్తుంది. దాని కోసం అది **2,00,000 ఆటగాళ్ళని sort** చేస్తుంది – ఆపై ఆ sorted జాబితాని విసిరేస్తుంది. తర్వాతి call మళ్ళీ అదే చేస్తుంది.

మౌలిక తప్పు: క్రమం (order) అనేది ఈ system యొక్క ముఖ్యమైన స్థితి – కానీ మనం దాన్ని నిల్వ చేయట్లేదు. ప్రతిసారీ తిరిగి లెక్కిస్తున్నాం.

Deep Dive 12 §4 lo సరిగ్గా ఇదే ఆకారం చూశాం: నిర్మాణాన్ని నిల్వ చేయకపోతే, ప్రతి ప్రశ్నకీ దాన్ని తిరిగి కనుక్కోవాలి.

5. Step – జాబితాని ఎప్పుడూ క్రమంలోనే ఉంచడం

క్రమాన్ని నిర్వహిద్దాం. Scores దిగువకి క్రమంలో ఒక array, మరియు rank కోసం **binary search**:

```
class SortedLeaderboard {
  constructor() {
    this.scores = new Map(); // player → score
    this.list = []; // [score, player] – ఎక్కువ నుంచి తక్కువకి
  }

  // ఒక score కంటే *ఖచ్చితంగా* ఎక్కువ* ఎంత మందికి ఉందో – binary search
  #countAbove(score) {
    let lo = 0, hi = this.list.length;
    while (lo < hi) {
      const mid = (lo + hi) >> 1;
      if (this.list[mid][0] > score) lo = mid + 1; else hi = mid;
    }
    return lo;
  }
}
```

```

    }

    top(k) { return this.list.slice(0, k).map([s, p] => [p, s]); }

    rank(player) {
      const s = this.scores.get(player);
      return s === undefined ? -1 : this.#countAbove(s) + 1;
    }
  }
}

```

ఆ **#countAbove** ఒక ముఖ్యమైన ఆలోచన. "Ravi జాబితాలో ఎక్కడ ఉన్నాడు?" అని వెతకట్లేదు – "Ravi కంటే ఎక్కువ score ఎంత మందికి ఉంది?" అని **లెక్కిస్తున్నాం**. జవాబు ఒకటే, కానీ రెండోది Ravi ఎవరో తెలియకుండానే లెక్కించగలదు. §8 lo ఇదే ఆలోచన మొత్తం పరిష్కారానికి దారితీస్తుంది.

Reads ఇప్పుడు:

ఆటగాళ్ళు	2,000 reads	
	sort-each	sorted
1000	183 ms	→ 1 ms
20000	6682 ms	→ 1 ms
200000	105605 ms	→ 1 ms

కానీ ఒక క్షణం – ఈ రెండు **VERSION** లూ ఒక జవాబు ఇస్తున్నాయా?

నేను ఆ పోలిక పెట్టాను, మరియు అవి **ఇవ్వట్లేదు**. 2,000 rank queries lo:

1,000 ఆటగాళ్ళు → 9 తేడాలు · 2,00,000 ఆటగాళ్ళు → 1,883 తేడాలు

కారణం: సమాన scores. పాత version `findIndex` వాడుతుంది – అంటే సమాన score ఉన్నవారికి వారు `array lo` ఎక్కడ పడ్డారో బట్టి వేర్వేరు ranks. కొత్తది `countAbove` వాడుతుంది – సమాన score → **సమాన rank**.

రెండోదే సరైనదని నేను అనుకుంటున్నాను. కానీ అది **నా అభిప్రాయం**, ఒక bug fix కాదు – మరియు ఆ నిర్ణయాన్ని ఇంకా ఎవరూ తీసుకోలేదు. **అదే Part 4 (§9-§11).**

1,05,605 ms → 1 ms. మరియు మళ్ళీ ఆ ముఖ్యమైన విషయం: tree వరుసలాగే, ఇక్కడ కూడా **1 ms అనేది మూడు సందర్భాల్లోనూ ఒకటే**. ఆటగాళ్ళ సంఖ్యతో సంబంధం లేదు.

విరుపు పరిష్కారమైంది. కానీ...

Part 3 – రెండో విరుపు: ఇప్పుడు రాయడం విరిగింది

6. Step – గడియారం ఇక్కడ అబద్ధం చెప్పింది

`submit` ఇప్పుడు ఏమి చేస్తోందో చూద్దాం:

```
submit(player, score) {  
  const old = this.scores.get(player);  
  if (old !== undefined) {  
    const i = this.#indexOf(player, old);  
    if (i !== -1) this.list.splice(i, 1); // ← O(n)  
  }  
  this.scores.set(player, score);  
  this.list.splice(this.#countAbove(score), 0, [score, player]); // ← O(n)  
}
```

రెండు `splice` calls. ప్రతి `splice` array మధ్యలో ఒక మూలకాన్ని తీసేయడం/చేర్చడం – అంటే **దాని తర్వాతి అన్ని మూలకాలనీ కదిలించడం**.

కొలుద్దాం... మరియు ఇక్కడే నేను ఇబ్బంది పడ్డాను:

2,000 submits	– వేర్వేరు runs, ఒకే code
n= 20000	89 ms
n= 100000	28 ms ← ఎక్కువ డేటా, తక్కువ సమయం?
n= 200000	53 ms
n= 500000	1861 ms

ఈ సంఖ్యలని ప్రచురించలేము

1,00,000 అటగాళ్ళకి **28 ms**, కానీ 20,000 కి **89 ms**? ఎక్కువ డేటాకి తక్కువ సమయమా? అది algorithm గురించి ఏమీ చెప్పదు.

కారణం V8 `splice` లోతుల్లో ఉంది – array representation, GC, memory layout. ఒకే code, ఒకే input – కానీ పరిమాణం మారితే **66 రెట్లు** అస్తవ్యస్తమైన తేడా.

ఇక్కడ చాలా మంది చేసే తప్పు: మళ్ళీ మళ్ళీ నడిపి, తమకి నచ్చిన సంఖ్యని తీసుకోవడం. అది కొలత కాదు, ఎంపిక.

సరైన పని: వేరే విషయాన్ని కొలవడం.

7. రెండో విరుపు – ఒక్క submit కి 2,00,560 మూలకాలు

గడియారానికి బదులు `splice` ఎన్ని మూలకాలని కదిలిస్తుందో లెక్కిద్దాం:

```
submit(player, score) {  
  const old = this.scores.get(player);  
  if (old !== undefined) {  
    const i = this.indexOf(player, old);  
    if (i !== -1) { this.moved += this.list.length - i; this.list.splice(i, 1); }  
  }  
  this.scores.set(player, score);  
  const j = this.countAbove(score);  
  this.moved += this.list.length - j; // ← పనిని లెక్కించడం  
  this.list.splice(j, 0, [score, player]);  
}
```

2,000 submits – splice ఎన్ని మూలకాలని కదిలించింది

ఆటగాళ్ళు	కదిలిన మూలకాలు	ఒక్కే submit కి
1000	19,83,807	992
20000	4,01,35,824	20,068
200000	40,11,20,653	2,00,560

రెండో విరుపు – చదవడం సరిచేసి, రాయడం విరగొట్టాం

ఒక్క score update కి 2,00,560 మూలకాలు కదులుతున్నాయి. అంటే సరిగ్గా n – ఆటగాళ్ళ సంఖ్య. $992 \rightarrow 20,068 \rightarrow 2,00,560$: ఖచ్చితంగా రేఖీయం.

మరియు గమనించండి – ఈ సంఖ్యలు **ప్రతి run lo అక్షరాలా ఒకటే**. గడియారం 28 ms నుంచి 1,861 ms వరకు ఊగింది; ఈ లెక్క ఊగలేదు, ఎందుకంటే **ఇదే నిజమైన పని**.

మౌలిక తప్పు: ఒక array క్రమాన్ని బాగా నిల్వ చేస్తుంది, కానీ **మధ్యలో మార్పులని** భరించదు. మరియు ఒక ఆటలో scores **నిరంతరం** మారుతాయి – అదే system చేసే ప్రధాన పని.

రెండు DESIGN లూ ఒక్కే వైపు విరుగుతున్నాయి

ఒకదాన్ని సరిచేస్తే రెండోది విరుగుతుంది – మనకి రెండూ కావాలి

ప్రతిసారీ sort (\$3)

రాయడం: **తక్షణం** ✓

చదవడం: 51 ms ✗

క్రమంలో ఉంచిన array (\$5)

చదవడం: **1 ms** ✓

రాయడం: 2,00,560 కదలికలు ✗

కావాల్సింది: రెండూ log వేగంతో

Score ప్రకారం **లెక్కించగలిగే** ఒక structure –

ఆటగాళ్ళని క్రమంలో ఉంచనిది. → **Fenwick tree (\$8)**

8. Step – Fenwick tree · score ప్రకారం లెక్కించడం

§5 lo ఒక ఆలోచన వచ్చింది: "Ravi ఎక్కడ ఉన్నాడు" అని వెతకొద్దు – "Ravi కంటే ఎక్కువ ఎంత మందికి?" అని లెక్కించు.

ఆ ఆలోచనని పూర్తిగా అనుసరిద్దాం. **అటగాళ్ళని క్రమంలో ఉంచడం అవసరమే లేదు.** కావాల్సింది ఒక్కటే: "1500 కంటే ఎక్కువ score ఎంత మందికి ఉంది?"

అంటే – ప్రతి score కి **ఎంత మంది ఉన్నారో** ఒక గణన, మరియు ఆ గణనల **మొత్తాన్ని వేగంగా** తీసే సామర్థ్యం. అదే **Fenwick tree** (Binary Indexed Tree).

```
class Fenwick {
    constructor(size) { this.n = size; this.t = new Int32Array(size + 1); }
    add(i, d) { for (i++; i <= this.n; i += i & -i) this.t[i] += d; }
    prefix(i) { let s = 0; for (i++; i > 0; i -= i & -i) s += this.t[i]; return s; }
}
```

ఆ $i \& -i$ ఏమిటి? అది i యొక్క **అతి కుడి 1-బిట్**. Fenwick tree lo ప్రతి index ఒక పరిధి యొక్క మొత్తాన్ని పట్టుకుంటుంది, మరియు ఆ పరిధి పొడవు సరిగ్గా ఆ బిట్. కాబట్టి $i += i \& -i$ అంటే "నన్ను కలుపుకునే తర్వాతి పెద్ద పరిధికి వెళ్ళు", మరియు $i -= i \& -i$ అంటే "నా పరిధిని లెక్కించి, దాని ముందున్నదానికి వెళ్ళు".

మీరు దీన్ని **గుర్తుపెట్టుకోవాలి**, రాబట్టాల్సిన అవసరం లేదు – నాలుగు పంక్తులు, మరియు అవి ఎప్పుడూ ఇలాగే ఉంటాయి. Interview lo ముఖ్యమైనది **ఎందుకు దీన్ని ఎంచుకున్నారో**, ఎలా అమలు చేశారో కాదు.

ఇప్పుడు leaderboard:

```
class FastLeaderboard {
    constructor(maxScore = 100000) {
        this.max = maxScore;
        this.scores = new Map(); // player → score
        this.byScore = new Map(); // score → Set<player>
        this.counts = new Fenwick(maxScore + 1);
        this.total = 0;
    }
    submit(player, score) {
        const old = this.scores.get(player);
        if (old !== undefined) { this.byScore.get(old).delete(player);
                                this.counts.add(old, -1); }
        else this.total++;
        this.scores.set(player, score);
        if (!this.byScore.has(score)) this.byScore.set(score, new Set());
        this.byScore.get(score).add(player);
        this.counts.add(score, +1);
    }
    countAbove(score) { return this.total - this.counts.prefix(score); }
    rank(player) {
        const s = this.scores.get(player);
        return s === undefined ? -1 : this.countAbove(s) + 1;
    }
}
```

Fenwick గణనలని పట్టుకుంటుంది; **byScore** పేర్లని పట్టుకుంటుంది. ఏ ఆటగాడూ ఏ క్రమంలోనూ లేడు.

మళ్ళీ అదే లెక్క

2,000 submits – Fenwick ఎన్ని అడుగులు వేసింది

ఆటగాళ్ళు	అడుగులు	ఒక్కో submit కి
1000	35,139	17.6
20000	35,154	17.6
200000	35,079	17.5

2,00,560 → 17.6

మూడు సందర్భాల్లోనూ **~17.6 అడుగులు**. ఆటగాళ్ళ సంఖ్యతో సంబంధమే లేదు – ఎందుకంటే Fenwick యొక్క ఖర్చు **ఆటగాళ్ళ సంఖ్య మీద కాదు, score పరిధి మీద** ఆధారపడుతుంది.

(17.6 ఎక్కడ నుంచి? ఒక **submit** = రెండు **add** calls, ఒక్కొక్కటి సగటున ~8.8 అడుగులు. Score పరిధి 1,00,000 కి $\log_2 \approx 16.6$, మరియు సగటు దానిలో సగం.)

ఇదే ఈ problem యొక్క కేంద్ర ఆలోచన: కోటి మంది ఆటగాళ్ళు ఉండొచ్చు, కానీ **scores లక్ష మాత్రమే**. కాబట్టి scores ని index చేయాలి, ఆటగాళ్ళని కాదు.

రెండు ఒకే జవాబులు ఇస్తున్నాయా?

వేగం అర్థవంతం కావాలంటే జవాబు సరైనదై ఉండాలి. 20,000 యాదృచ్ఛిక operations, ప్రతి 200 అడుగులకీ రెండు version లని పోల్చడం:

20,000 operations · రెండు ఒకే జవాబులు ఇచ్చాయా? అవును ✓

Part 4 – మూడో విరుపు: సమాన scores

9. Step – ముగ్గురికీ 1500 • ఎవరు ముందు?

ఇప్పుడు వేగం బాగుంది. ఒక సాధారణ leaderboard చూద్దాం – ముగ్గురికీ సరిగ్గా 1500:

మొదట:

1.	asha	1850
2.	ravi	1500
2.	meera	1500
2.	kiran	1500
5.	sai	900

Ranks **1, 2, 2, 2, 5** – ముగ్గురికీ ఒకే rank, ఆపై 5 కి దూకుతుంది. దీన్ని **పోటీ ర్యాంకింగ్** (competition ranking) అంటారు, ఒలింపిక్స్ లో వాడేది.

ఇది సరైనదే. కానీ ప్రదర్శన క్రమం చూడండి – ravi, meera, kiran. ఆ క్రమం ఎక్కడి నుంచి వచ్చింది?

పక్కదారి – నా **BENCHMARK** డేటా అబద్ధం చెప్తోంది

"ఎంత మందికీ సమాన score ఉంటుంది?" అని కొలిచాను. జవాబు: 20,000 ఆటగాళ్ళకి **87%**. అది చాలా ఎక్కువ – 20,000 మందిని 1,00,000 scores లో పంచితే ~10% మాత్రమే ఢీ కొట్టాలి.

తప్పు నా random number generator లో ఉంది. నేను ఒక సరళమైన LCG వాడాను ($seed = seed * a + c \text{ mod } 2^{31}$). అది 20,000 నమూనాల్లో **13,499 విభిన్న విలువలు** ఇచ్చింది; సరైన generator **18,136** ఇచ్చింది – ఆశించినది 18,127.

కాబట్టి ఈ doc లో ఉన్న ప్రతి సంఖ్యనీ మంచి generator (mulberry32) తో మళ్ళీ కొలిచాను. వేగపు ఫలితాలు పెద్దగా మారలేదు – కానీ **ties** గురించిన ప్రతి ముగింపు మారేది.

పాఠం: ఒక benchmark యొక్క డేటా కూడా ఒక ఊహ. అది ఎంత నిజమో ముందు తనిఖీ చేయండి – ఫలితాలు వచ్చాక కాదు.

10. మూడో విరుపు – ఏమీ మారకుండానే స్థానాలు మారాయి

meera మళ్ళీ ఆడింది మరియు సరిగ్గా అదే 1500 సాధించింది. ఏమీ మారలేదు:

```
lb.submit('meera', 1500); // అదే score
```

meera *అదే* 1500 మళ్ళీ submit చేశాక:

1. asha 1850
2. ravi 1500
2. kiran 1500
2. meera 1500
5. sai 900

మూడో ఎవరు – MEERA పైకి ఎక్కలేదు, అయినా కిందకి పడింది

meera ఏమీ సాధించలేదు, ఏమీ పోగొట్టుకోలేదు – అదే score మళ్ళీ submit చేసింది. అయినా ఆమె జాబితాలో రెండో నుంచి మూడో స్థానానికి జారింది, మరియు kiran కారణం లేకుండా పైకి వచ్చాడు.

ఎందుకు? byScore ఒక Set . meera ని తీసేసి మళ్ళీ చేర్చాం, కాబట్టి ఆమె Set చివరికి వెళ్ళింది. ఆ క్రమమే జాబితాలో కనిపిస్తోంది.

ఇది ఒక performance bug కాదు – ఇది ఒక నమ్మకం సమస్య. ఆటగాళ్ళు leaderboard ని చూస్తూ ఉంటారు. స్థానాలు కారణం లేకుండా మారితే వారు "ఏదో మోసం జరుగుతోంది" అనుకుంటారు – మరియు వారు తప్పు కాదు: జవాబు నిజంగా యాదృచ్ఛికమే.

అసలు ప్రశ్న: సమానంగా ఉంటే ఎవరు ముందు?

ఇది సాంకేతిక ప్రశ్న కాదు – ఇది ఆట యొక్క నియమం. మూడు సాధారణ జవాబులు:

నియమం	ఎవరు వాడతారు	ఇక్కడ ముగ్గురికి
ముందు చేరుకున్నవాడు ముందు	చాలా ఆటలు, LeetCode contests	ఎవరు ముందు 1500 చేరారో వారు
మరో కొలత (తక్కువ సమయం, తక్కువ ప్రయత్నాలు)	పరుగు పందాలు, speedruns	ఆ రెండో కొలత ప్రకారం
అక్షర క్రమం	సరళమైనది, కానీ అన్యాయం	asha, kiran, meera

మొదటిది సహజమైనది మరియు న్యాయమైనది: "నువ్వు ముందు ఆ score చేరుకున్నావు, కాబట్టి నువ్వే ముందు."

11. Step – ఎప్పుడు చేరుకున్నాడో గుర్తుపెట్టడం

ప్రతి ఆటగాడు తన ప్రస్తుత score ని ఎప్పుడు చేరుకున్నాడో ఒక వరుస సంఖ్య (seq) గా నిల్వ చేద్దాం:

```
submit(player, score) {
  const old = this.entry.get(player);
  if (old && old.score === score)
    return { ok: true, changed: false }; // ఏమీ మారలేదు → ఏమీ చేయొద్దు
  ...
  const seq = ++this.#seq; // ఈ score ని ఎప్పుడు చేరుకున్నాడు
  this.entry.set(player, { score, seq });
  this.byScore.get(score).set(player, seq);
}
```

```
...  
}
```

రెండు మార్పులు, మరియు **రెండూ ముఖ్యం**:

1 • changed: false – score మారకపోతే **ఏమీ చేయకూడదు**. Fenwick ని తాకొద్దు, seq ని మార్చొద్దు. ఇది §10 విరుపుని నేరుగా ఆపుతుంది.

ఇది ఒక సార్వత్రిక నియమం: ఏమీ మార్చని ఒక *update*, ఏమీ మార్చకూడదు. చాలా systems lo bugs ఇక్కడే పుడతాయి – "save" నొక్కితే `updated_at` మారుతుంది, cache ఖాళీ అవుతుంది, event publish అవుతుంది... **ఏమీ మారకపోయినా**.

2 • seq – సమాన scores ఉన్నవారి మధ్య ఒక **నిర్ణయాత్మకమైన** క్రమం. Set కి బదులు `Map<player, seq>`, మరియు చూపించేటప్పుడు `seq` ప్రకారం:

```
#bucket(score) {  
  return [...(this.byScore.get(score) ?? new Map())]  
    .sort((a, b) => a[1] - b[1]).map(([p]) => p);  
}
```

అదే పరీక్ష మళ్ళీ:

```
--- meera *అదే* 1500 మళ్ళీ submit ---  
{ ok: true, changed: false }  
క్రమం:  
1.  asha      1850  
2.  ravi      1500  
2.  meera     1500  
2.  kiran     1500  
5.  sai       900
```

meera అక్కడే ఉంది. మరియు `changed: false` అనేది call చేసినవారికి **స్పష్టంగా చెప్తోంది** – ఇది ఒక no-op అని. UI దాన్ని చూసి animation ని దాటయొచ్చు; analytics దాన్ని లెక్కించకుండా ఉండొచ్చు.

Part 5 – పూర్తి system

12. Step – "నేను ఎక్కడ ఉన్నాను" • మరియు నేనే మళ్ళీ చేసిన తప్పు

ప్రతి ఆటలో ఉండే feature: "మీరు #75,906 – మీ చుట్టుపక్కల వాళ్ళు ఇదిగో." ఇది ఆటగాళ్ళు ఆడిస్తూ ఉంచే విషయం.

నేను మొదట ఇలా రాశాను:

```
around(player, radius = 2) {  
  const e = this.entry.get(player);  
  if (!e) return { ok: false, reason: `NOT_FOUND: ${player}` };  
  const mine = this.slice(1, this.total).findIndex(r => r.player === player) + 1;  
  return { ok: true, rows: this.slice(mine - radius, mine + radius) };  
}
```

ఇది సరైన జవాబు ఇస్తుంది. కానీ ఆ `this.slice(1, this.total)` చూడండి – అది మొత్తం 2,00,000 వరుసలను కడుతుంది, ఆపై అందులో ఒక్క పేరు కోసం వెతుకుతుంది.

§4 ఎరువుని నేనే మళ్ళీ, నా స్వంత "సరిచేసిన" CODE LO

మూడు దశలు పెట్టి O(n) ని తొలగించాను – ఆపై ఒక సౌకర్యవంతమైన method రాసి దాన్ని వెనక్కి తెచ్చాను.

మరియు అది సరైన జవాబు ఇస్తుంది, కాబట్టి ఏ test ఫెయిల్ కాదు. Tests సరైనతని పరీక్షిస్తాయి, ఖర్చుని కాదు.

సరైన రూపం: స్థానాన్ని లెక్కించడం – §5 lo మొదలైన అదే ఆలోచన:

```
#positionOf(player) {  
  const e = this.entry.get(player);  
  if (!e) return -1;  
  const startedAt = this.countAbove(e.score) + 1; // నా బకెట్ ఎక్కడ మొదలు  
  return startedAt + this.#bucket(e.score).indexOf(player);  
}  
around(player, radius = 2) {  
  const pos = this.#positionOf(player);  
  if (pos === -1) return { ok: false, reason: `NOT_FOUND: ${player}` };  
  return { ok: true, rows: this.slice(pos - radius, pos + radius) };  
}
```

ఒక around() call:

మొత్తం జాబితా కట్టితే : 39.8 ms

స్థానం లెక్కిస్తే : 0.0008 ms

తేడా : 49,750x

13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

నిర్మాణం · ఏ ఆటగాడూ క్రమంలో లేడు

మాడు నిల్వలు - ఒక్కొక్కటి ఒక వేర్వేరు ప్రశ్నకి జవాబు



```
'use strict';
class Fenwick {
  constructor(size) { this.n = size; this.t = new Int32Array(size + 1); }
  add(i, d) { for (i++; i <= this.n; i += i & -i) this.t[i] += d; }
  prefix(i) { let s = 0; for (i++; i > 0; i -= i & -i) s += this.t[i]; return s; }
  // prefix(idx) >= target అయ్యే అతి చిన్న idx - binary lifting
  lowerBound(target) {
    let pos = 0, rem = target;
    for (let pw = 1 << 31 - Math.clz32(this.n); pw > 0; pw >= 1)
      if (pos + pw <= this.n && this.t[pos + pw] < rem) {
        pos += pw; rem -= this.t[pos];
      }
    return pos;
  }
}

class Leaderboard {
  #seq = 0;
  constructor(maxScore = 100000) {
    this.max = maxScore;
    this.entry = new Map(); // player → { score, seq }
    this.byScore = new Map(); // score → Map<player, seq>
    this.counts = new Fenwick(maxScore + 1);
    this.total = 0;
  }

  submit(player, score) {
    if (score < 0 || score > this.max)
      return { ok: false, reason: `OUT_OF_RANGE: ${score}` };
    const old = this.entry.get(player);
    if (old && old.score === score)
      return { ok: true, changed: false }; // ఏమీ మారలేదు → ఏమీ చేయొద్దు
    if (old) {
      const b = this.byScore.get(old.score);
      b.delete(player);
      if (!b.size) this.byScore.delete(old.score);
      this.counts.add(old.score, -1);
    } else this.total++;
  }
}
```

```

const seq = ++this.#seq; // ఈ score ని ఎప్పుడు చేరుకున్నాడు
this.entry.set(player, { score, seq });
if (!this.byScore.has(score)) this.byScore.set(score, new Map());
this.byScore.get(score).set(player, seq);
this.counts.add(score, +1);
return { ok: true, changed: true, rank: this.rank(player) };
}

remove(player) {
  const e = this.entry.get(player);
  if (!e) return { ok: false, reason: `NOT_FOUND: ${player}` };
  const b = this.byScore.get(e.score);
  b.delete(player);
  if (!b.size) this.byScore.delete(e.score);
  this.counts.add(e.score, -1);
  this.entry.delete(player);
  this.total--;
  return { ok: true };
}

score(player) { return this.entry.get(player)?.score ?? null; }
countAbove(score) { return this.total - this.counts.prefix(score); }

// పోటీ ర్యాంకు: సమాన score → సమాన rank (1, 2, 2, 2, 5)
rank(player) {
  const e = this.entry.get(player);
  return e ? this.countAbove(e.score) + 1 : -1;
}

// ఒక బకెట్‌లోని ఆటగాళ్ళు - ముందు చేరుకున్నవారు ముందు
#bucket(score) {
  return [...(this.byScore.get(score) ?? new Map())]
    .sort((a, b) => a[1] - b[1]).map(([p]) => p);
}

// i-వ స్థానం (1 నుంచి) ఉన్న ఆటగాడి score
#scoreAtPosition(i) { return this.counts.lowerBound(this.total - i + 1); }

// [from, to] స్థానాల మధ్య ఉన్నవారు (1 నుంచి)
slice(from, to) {
  const out = [];
  from = Math.max(1, from);
  to = Math.min(this.total, to);
  let pos = from;
  while (pos <= to) {
    const s = this.#scoreAtPosition(pos);
    const players = this.#bucket(s);
    const startedAt = this.countAbove(s) + 1; // ఈ బకెట్ ఎక్కడ మొదలు
    for (let j = pos - startedAt; j < players.length && pos <= to; j++, pos++)
      out.push({ rank: startedAt, position: pos, player: players[j], score: s });
  }
  return out;
}

top(k) { return this.slice(1, k); }

// నా స్థానం - మొత్తం జాబితా కట్టకుండా
#positionOf(player) {
  const e = this.entry.get(player);
  if (!e) return -1;
  const startedAt = this.countAbove(e.score) + 1;

```

```

        return startedAt + this.#bucket(e.score).indexOf(player);
    }
    around(player, radius = 2) {
        const pos = this.#positionOf(player);
        if (pos === -1) return { ok: false, reason: `NOT_FOUND: ${player}` };
        return { ok: true, rows: this.slice(pos - radius, pos + radius) };
    }
}

```

--- Top 5 ---

```

1. asha      1850
2. ravi      1500
2. meera     1500
2. kiran     1500
5. sai       900

```

--- meera *అదే* 1500 మళ్ళీ submit ---

```
{ ok: true, changed: false }
```

క్రమం:

```

1. asha      1850
2. ravi      1500
2. meera     1500
2. kiran     1500
5. sai       900

```

--- ravi 1900 కి పెరిగాడు ---

```
{ ok: true, changed: true, rank: 1 }
```

క్రమం:

```

1. ravi      1900
2. asha      1850
3. meera     1500
3. kiran     1500
5. sai       900

```

--- తిరస్కరణలు ---

```
score పరిధి దాటితే : { ok: false, reason: 'OUT_OF_RANGE: 999999' }
```

```
లేని ఆటగాడు       : -1
```

2,00,000 ఆటగాళ్ళతో:

2,00,000 ఆటగాళ్ళు

```
ravi rank : 77,661
```

```
2,000 rank calls : 0 ms
```

```
2,000 top(10)    : 5 ms
```

```
2,000 around()   : 3 ms
```

ravi చుట్టుపక్కల:

```

77660. p69430    61235
77661. p70783    61234
77661. ravi      61234
77663. p89984    61233
77663. p124519   61233

```

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

`changed: false` – meera అక్కడే ఉంది. §10 విరుపు మాయమైంది.

ravi 1900 కి పేరిగాక ranks 1, 2, 3, 3, 5 – meera మరియు kiran ఇద్దరికీ rank 3 (వారి కంటే ఇద్దరు ఎక్కువ), sai కి 5 (నలుగురు ఎక్కువ). **గణితం సరైనది, మరియు క్రమం నిలకడగా ఉంది.**

77661. p70783 మరియు 77661. ravi – ఇద్దరికీ 61234, ఇద్దరికీ **ఒకే rank**. కానీ ప్రదర్శనలో p70783 ముందు ఉన్నాడు, ఎందుకంటే అతను **ముందు** ఆ score చేరుకున్నాడు. `rank` సమానం, `position` వేరు – **రెండూ విడిగా ఉన్నాయి**, మరియు అదే సరైనది.

77663 కి దూకింది – 77661, 77661 తర్వాత 77662 కాదు, **77663**. పోటీ ర్యాంకింగ్ అంటే అదే.

rank 2,000 calls కి 0 ms – 2,00,000 ఆటగాళ్ళలో. §4 lo ఒక్క call కి 51 ms.

ఆ `top` మరియు `around` సంఖ్యలు runs మధ్య 2-11 ms ఉుగుతాయి. §6 lo చెప్పినట్టే – **ఇంత చిన్న సమయాలని నమ్మొద్దు**. ఇక్కడ ముఖ్యమైనది ఒక సంఖ్య కాదు: **2,00,000 ఆటగాళ్ళ మీద మూడు operations ఒక్కో దానికీ మిల్లీసెకనులో ముగుస్తున్నాయి**, §4 lo ఒక్క దానికి 51 ms పట్టేది.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	<code>Map<player, score></code>	మౌలిక అస్థిపంజరం
§4 (విరుపు)	క్రమాన్ని నిల్వ చేయడం	ఒక్క read కి 51 ms
§5	<code>countAbove</code> – వెతకడం కాదు, లెక్కించడం	ఈ ఆలోచన మిగతా అంతటికీ ఆధారం
§7 (విరుపు)	<code>Fenwick</code>	ఒక్క submit కి 2,00,560 కదలికలు
§8	<code>byScore</code> విడిగా	గణనలు మరియు పేర్లు వేర్వేరు పనులు
§10 (విరుపు)	<code>seq</code> + <code>changed: false</code>	ఏమీ మారకుండా leaderboard తారుమారైంది
§12	<code>#positionOf</code>	నా స్వంత <code>around</code> $O(n)$ ని వెనక్కి తెచ్చింది
§13	<code>rank</code> మరియు <code>position</code> విడిగా	సమాన rank, వేర్వేరు స్థానాలు

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

Fenwick కి ఎక్కువ సమయం ఇవ్వండి - మిగతాది వేగంగా

5 నిమి

Clarify – top-K నా rank నా? ties? score పరిధి?

8 నిమి

Naive → sorted → రెండూ ఎందుకు విఫలం

14 నిమి – Fenwick

scores ని index చేయడం · ఇదే కేంద్రం

9 నిమి – Ties

seq · changed:false · rank vs position

9 నిమి

around · Redis · కాలపరిమితి · విభజన

ఏమి తప్పక చెప్పాలి

1. "Top 10" మరియు "నా rank" వేర్వేరు problems (§1) – మరియు heap రెండోదానికి పనికిరాదు. ఇది మొదటి 60 సెకన్లలో చెప్పండి.
2. అటగాళ్ళని కాదు, scores ని index చెయ్యి (§8) – "కోటి అటగాళ్ళు, కానీ లక్ష scores." ఇదే మొత్తం ఆలోచన.
3. వెతకొద్దు, లెక్కించు (§5) – countAbove . Rank అంటే "నా కంటే ఎక్కువ ఎంత మంది" + 1.
4. Ties ని మీరే లేవనెత్తండి (§10) – మరియు "ఏమీ మార్చని update ఏమీ మార్చకూడదు" అనే నియమం.
5. rank మరియు position వేర్వేరు (§13) – సమాన rank, వేర్వేరు ప్రదర్శన స్థానాలు.

ఏమి వదిలేయాలి

- Fenwick ని పూర్తిగా రాయొద్దు – add మరియు prefix నాలుగు పంక్తులు రాసి, "binary lifting తో k-వ మూలకాన్ని కూడా తీయొచ్చు" అని చెప్పండి. lowerBound ని రాయొద్దు.
- slice పూర్తిగా రాయొద్దు – top మరియు around రెండూ ఇదే అని చెప్తే చాలు.
- Naive version lo ఎక్కువ సమయం పెట్టొద్దు – రెండు నిమిషాలు, ఆపై "ఇది $O(n \log n)$ per read, కొలిస్తే 2 లక్షల అటగాళ్ళకి 51 ms" అని ముందుకి వెళ్ళండి.
- Balanced BST / skip list గురించి మాట్లాడండి, రాయొద్దు (§16).

15. నీటితో చెప్పాల్సిన English script

"First – which question dominates: 'who's in the top ten' or 'what's my rank'? They look similar and they're completely different problems. And two more: what's the score range, and what happens on a tie?

I ask about top-K versus rank because the textbook answer for top-K is a size-K min-heap, and that's the wrong tool here. A heap knows its top; it has no idea where an arbitrary player sits. In a real game ten people are in the top ten and ten million people are checking their own rank, so rank is the operation to design around.

The obvious version keeps a map of player to score and sorts on every read. That works and it's five lines, but every read redoes the whole ordering. I measured it – one rank lookup over two hundred thousand players took about forty-eight milliseconds, so ten thousand of them is eight minutes of CPU for one second of traffic.

The next instinct is to keep the list sorted, and reads do get fast – binary search gives you the count of players above a score, which is the rank. But now every score update has to splice into the middle of the array. I tried to time it and the numbers were useless – the same code ranged from twenty-eight milliseconds to thirteen hundred across runs, because of how V8 handles array internals. So instead of timing it I counted the work: how many elements does splice move? That number is identical every run – about two hundred thousand elements moved per submit at two hundred thousand players. Exactly linear, which is what you'd expect, and in a game scores are changing constantly.

So neither ordering strategy works, because I was ordering the wrong thing. There might be ten million players, but there are only a hundred thousand distinct scores. So I'd index the scores, not the players: a Fenwick tree over the score range holding how many players have each score. A rank is then the total minus the prefix count up to your score, plus one – no player is stored in any order at all. Same counting experiment gives about seventeen steps per submit, and crucially that's the same number at a thousand players and at two hundred thousand, because the cost depends on the score range, not the player count. I'd keep a separate map from score to the players holding it, so I can turn positions back into names.

On ties – this is worth raising before they ask. If tied players are stored in a set and someone resubmits the same score, they get removed and reinserted, and they move in the display even though nothing happened. Players watching the board see themselves drop for no reason and assume it's rigged. Two fixes: first, an update that doesn't change the score should do nothing at all – return early. Second, record a sequence number for when a player reached their current score, and order within a tie by that, so whoever got there first shows first.

I'd also keep rank and display position as separate fields. Tied players genuinely share a rank – that's competition ranking, one two two two five – but they still need a stable order on screen. Conflating those is where most tie bugs come from."

16. Follow-ups – Redis, కాలపరిమితి, విభజన

Follow-up	జవాబు	మూరే classes
"Scores పరిధి చాలా పెద్దది (దశాంశాలు, కోట్లు)"	Fenwick పరిమాణం పెరుగుతుంది → balanced BST లేదా skip list వాడాలి	counts
"రోజువారీ / వారపు leaderboard"	పలు Leaderboard వస్తువులు , ఒక్క కాలానికి ఒకటి	0 concepts
"ఒక ఆటగాడిని ban చెయ్యి"	remove ఇప్పటికే ఉంది	0
"నా స్నేహితుల మధ్య నా rank"	స్నేహితుల scores తీసి sort – జాబితా చిన్నది	+1 method
"అత్యుత్తమ score మాత్రమే ఉంచాలి"	submit lo if (score <= old.score) return	1 పంక్తి
"నిజంగా production lo ఏమి వాడతారు?"	కింద చూడండి	–
"కోట్ల మంది · పలు servers"	కింద చూడండి	Storage layer

నిజ ప్రపంచంలో: Redis sorted set

"ఆచరణలో చాలా జట్లు దీన్ని స్వయంగా రాయవు – **Redis sorted set** (`ZADD` , `ZRANK` , `ZRANGE`) వాడతాయి. అది లోపల **skip list + hash map** – నేను చేసినదే, కానీ score పరిధి మీద ఎలాంటి హద్దు లేకుండా.

మరియు ties కి Redis ఒక తెలివైన ఉపాయం వాడుతుంది: score మరియు tiebreak ని **ఒకే సంఖ్యలో కలపడం** – $score * పెద్దది + (గరిష్ట_సమయం - timestamp)$. అప్పుడు structure కి ties అనేవే ఉండవు; అన్నీ విభిన్న విలువలే.

కానీ ఆ ఉపాయానికి ఒక ధర ఉంది: ఇప్పుడు `ZRANK` ఇచ్చేది **ordinal position**, పోటీ rank కాదు. సమాన score ఉన్నవారికి వేర్వేరు ranks వస్తాయి. §13 lo నేను వాటిని విడిగా ఉంచాను – ఏది కావాలో అది **ఆట యొక్క నిర్ణయం**.

Interview lo Redis పేరు చెప్పడం మంచిది – కానీ **ముందు లోపల ఏమి ఉందో చెప్పండి**. "Redis వాడతాను" అనేది design కాదు."

Scale

"Fenwick tree ఒక array – దాన్ని servers మధ్య పంచడం కష్టం.

ఆచరణలో: **score పరిధిని ముక్కలుగా విభజించడం** (sharding). ఒక్కో shard ఒక score పరిధిని పట్టుకుంటుంది మరియు తన మొత్తాన్ని తెలుసుకుంటుంది. Rank అంటే – నా shard lo లెక్కించి, నా పైన ఉన్న shards యొక్క మొత్తాలని కలపడం. Shards పది ఉంటే అది పది చిన్న calls.

మరియు ఒక ముఖ్యమైన సరళీకరణ: rank కి **ఖచ్చితత్వం అవసరమా?** "#75,906" మరియు "#75,910" మధ్య ఏ అటగాడికీ తేడా తెలియదు. కాబట్టి పెద్ద leaderboards lo **top 1,000 ఖచ్చితంగా**, మిగతావి **ఉజ్జాయింపుగా** – ఇది ఖర్చుని భారీగా తగ్గిస్తుంది.

ఈ ఆలోచన – **ఎక్కడ ఖచ్చితత్వం అవసరం లేదో గుర్తించడం** – interview lo చాలా బలమైన సంకేతం."

17. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
వెతకొడ్డు – లెక్కించు	<code>countAbove</code> (§5, §8)	Deep Dive 08, 10 §8, 11 §8, 12 §5
సరైన విషయాన్ని index చెయ్యి	అటగాళ్ళని కాదు, scores ని (§8)	Databases, search, Deep Dive 11
Benchmark అస్థిరమైతే వేరే కొలత	28-1,861 ms → 2,00,560 కదలికలు (§7)	ప్రతి optimisation
ఏమే మార్పుని update, ఏమే మార్చకూడదు	<code>changed: false</code> (§11)	Caches, events, <code>updated_at</code> , re-render
నిర్ణయాత్మక క్రమం ఒక feature	<code>seq</code> (§11)	Pagination, sync, diffs
ఒకే డేటా, రెండు ప్రశ్నలు	<code>rank</code> vs <code>position</code> (§13)	UI vs domain modelling
సాకర్వచంతమైన method ఖర్చుని దాచేస్తుంది	నా <code>around</code> (§12)	ORMs, getters, helper functions

ఒక చివరి ఆలోచన – ఈ **DOC LO** నేను రెండుసార్లు తప్పు చేశాను

ఒకసారి (§6): ఒక benchmark రాసి, దాని సంఖ్యలు అర్థరహితమని గమనించాను – 1,00,000 అటగాళ్ళకి 20,000 కంటే తక్కువ సమయం. వాటిని ప్రచురిస్తే మీరు ఒక తప్పు నమ్మేవారు.

రెండోసారి (§12): మొత్తం doc `O(n)` ని తొలగించడం గురించి రాసి, ఆపై చివరి method lo దాన్ని **వెనక్కి తెచ్చాను**. Tests అన్నీ pass అయ్యాయి, ఎందుకంటే జవాబు సరైనదే.

రెండూ ఒకే పాఠాన్ని చెప్తాయి: "పనిచేస్తోంది" మరియు "సరైనది" ఒకటి కాదు. Tests సరైనతని చూస్తాయి; ఖర్చుని మీరే చూడాలి – మరియు మీ స్వంత "సరిచేసిన" code ని కూడా.

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator · 07 Pub-Sub · 08 HashMap · 09 Chess · 10 Meeting Scheduler · 11 Food Delivery · 12 File System · 13 Leaderboard.**

వేగవంతమైన revision కోసం – [LLD_Design_Problems_Telugu.pdf](#) .

Leaderboard – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, ప్రతి సంఖ్య నిజంగా node lo run చేసి తీసినదే ✓